第3章　线性模型

正确的判断来自于经验，而经验来自于错误的判断.

——弗雷德里克·布鲁克斯（Frederick P. Brooks）
1999年图灵奖获得者

线性模型（Linear Model）是机器学习中应用最广泛的模型，指通过样本特征的线性组合来进行预测的模型. 给定一个 D 维样本 $\boldsymbol{x} = [x_1, \cdots, x_D]^{\mathrm{T}}$，其线性组合函数为

$$f(\boldsymbol{x}; \boldsymbol{w}) = w_1 x_1 + w_2 x_2 + \cdots + w_D x_D + b \tag{3.1}$$

$$= \boldsymbol{w}^{\mathrm{T}} \boldsymbol{x} + b, \tag{3.2}$$

为简单起见，这里我们用 $f(\boldsymbol{x}; \boldsymbol{w})$ 来表示 $f(\boldsymbol{x}; \boldsymbol{w}, b)$.

其中 $\boldsymbol{w} = [w_1, \cdots, w_D]^{\mathrm{T}}$ 为 D 维的权重向量，b 为偏置. 上一章中介绍的线性回归就是典型的线性模型，直接用 $f(\boldsymbol{x}; \boldsymbol{w})$ 来预测输出目标 $y = f(\boldsymbol{x}; \boldsymbol{w})$.

在分类问题中，由于输出目标 y 是一些离散的标签，而 $f(\boldsymbol{x}; \boldsymbol{w})$ 的值域为实数，因此无法直接用 $f(\boldsymbol{x}; \boldsymbol{w})$ 来进行预测，需要引入一个非线性的决策函数（Decision Function）$g(\cdot)$ 来预测输出目标

$$y = g\big(f(\boldsymbol{x}; \boldsymbol{w})\big), \tag{3.3}$$

其中 $f(\boldsymbol{x}; \boldsymbol{w})$ 也称为判别函数（Discriminant Function）.

对于二分类问题，$g(\cdot)$ 可以是符号函数（Sign Function），定义为

$$g\big(f(\boldsymbol{x}; \boldsymbol{w})\big) = \operatorname{sgn}\big(f(\boldsymbol{x}; \boldsymbol{w})\big) \tag{3.4}$$

$$\triangleq \begin{cases} +1 & \text{if} \quad f(\boldsymbol{x}; \boldsymbol{w}) > 0, \\ -1 & \text{if} \quad f(\boldsymbol{x}; \boldsymbol{w}) < 0. \end{cases} \tag{3.5}$$

当 $f(\boldsymbol{x}; \boldsymbol{w}) = 0$ 时不进行预测. 公式(3.5)定义了一个典型的二分类问题的决策函数，其结构如图3.1所示.

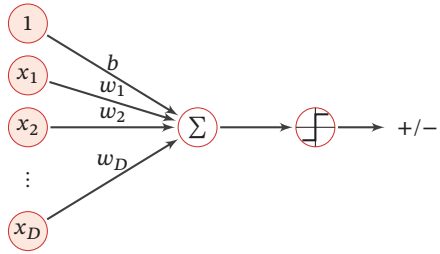


图 3.1 二分类的线性模型

在本章，我们主要介绍四种不同线性分类模型：Logistic 回归、Softmax 回归、感知器和支持向量机，这些模型的区别主要在于使用了不同的损失函数.

3.1 线性判别函数和决策边界

从公式(3.3)可知，一个线性分类模型（Linear Classification Model）或线性分类器（Linear Classifier），是由一个（或多个）线性的判别函数 $f(\boldsymbol{x};\boldsymbol{w}) = \boldsymbol{w}^{\mathrm{T}}\boldsymbol{x} + b$ 和非线性的决策函数 $g(\cdot)$ 组成. 我们首先考虑二分类的情况，然后再扩展到多分类的情况.

3.1.1 二分类

二分类（Binary Classification）问题的类别标签 y 只有两种取值，通常可以设为 $\{+1, -1\}$ 或 $\{0, 1\}$. 在二分类问题中，常用正例（Positive Sample）和负例（Negative Sample）来分别表示属于类别 $+1$ 和 -1 的样本.

在二分类问题中，我们只需要一个线性判别函数 $f(\boldsymbol{x};\boldsymbol{w}) = \boldsymbol{w}^{\mathrm{T}}\boldsymbol{x} + b$. 特征空间 $\mathbb{R}^D$ 中所有满足 $f(\boldsymbol{x};\boldsymbol{w}) = 0$ 的点组成一个分割超平面（Hyperplane），称为决策边界（Decision Boundary）或决策平面（Decision Surface）. 决策边界将特征空间一分为二，划分成两个区域，每个区域对应一个类别.

超平面就是三维空间中的平面在更高维空间的推广. D 维空间中的超平面是 $D-1$ 维的. 在二维空间中，决策边界为一个直线；在三维空间中，决策边界为一个平面；在高维空间中，决策边界为一个超平面.

参见习题3-2.

所谓“线性分类模型”就是指其决策边界是线性超平面. 在特征空间中，决策平面与权重向量 $\boldsymbol{w}$ 正交. 特征空间中每个样本点到决策平面的有向距离（Signed Distance）为

$$\gamma = \frac{f(\boldsymbol{x};\boldsymbol{w})}{\|\boldsymbol{w}\|}. \tag{3.6}$$

γ 也可以看作点 $\boldsymbol{x}$ 在 $\boldsymbol{w}$ 方向上的投影.

图3.2给出了一个二分类问题的线性决策边界示例，其中样本特征向量 $\boldsymbol{x} = [x_1, x_2]$，权重向量 $\boldsymbol{w} = [w_1, w_2]$.

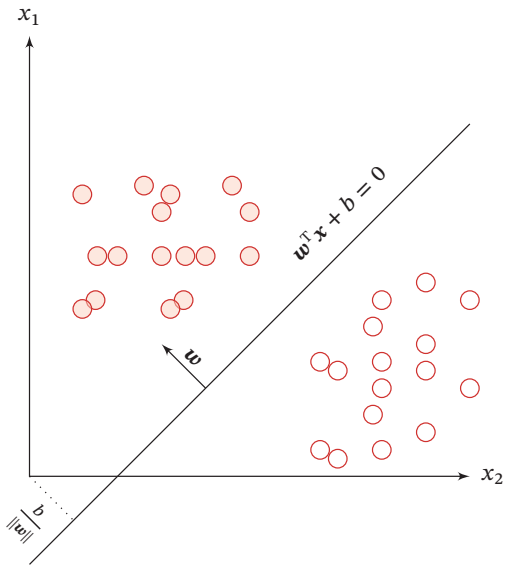


图 3.2 二分类的决策边界示例

给定 N 个样本的训练集 $\mathcal{D}=\{(\boldsymbol{x}^{(n)},y^{(n)})\}_{n=1}^{N}$，其中 $y^{(n)}\in\{+1,-1\}$，线性模型试图学习到参数 $\boldsymbol{w}^*$，使得对于每个样本 $(\boldsymbol{x}^{(n)},y^{(n)})$ 尽量满足

$$\begin{aligned} f(\boldsymbol{x}^{(n)};\boldsymbol{w}^*)>0 \quad &\text{if} \quad y^{(n)}=1, \\ f(\boldsymbol{x}^{(n)};\boldsymbol{w}^*)<0 \quad &\text{if} \quad y^{(n)}=-1. \end{aligned} \tag{3.7}$$

上面两个公式也可以合并，即参数 $\boldsymbol{w}^*$ 尽量满足

$$y^{(n)}f(\boldsymbol{x}^{(n)};\boldsymbol{w}^*)>0, \qquad \forall n\in[1,N]. \tag{3.8}$$

定义 3.1 – 两类线性可分：对于训练集 $\mathcal{D}=\{(\boldsymbol{x}^{(n)},y^{(n)})\}_{n=1}^{N}$，如果存在权重向量 $\boldsymbol{w}^*$，对所有样本都满足 $yf(\boldsymbol{x};\boldsymbol{w}^*)>0$，那么训练集 $\mathcal{D}$ 是线性可分的.

为了学习参数 $\boldsymbol{w}$，我们需要定义合适的损失函数以及优化方法. 对于二分类问题，最直接的损失函数为 0-1 损失函数，即

$$\mathcal{L}_{01}\big(y,f(\boldsymbol{x};\boldsymbol{w})\big)=I\big(yf(\boldsymbol{x};\boldsymbol{w})<0\big), \tag{3.9}$$

其中 $I(\cdot)$ 为指示函数. 但 0-1 损失函数的数学性质不好，其关于 $\boldsymbol{w}$ 的导数为 0，从而导致无法优化 $\boldsymbol{w}$.

3.1.2 多分类

多分类（Multi-class Classification）问题是指分类的类别数 C 大于 2. 多分类一般需要多个线性判别函数，但设计这些判别函数有很多种方式.

假设一个多分类问题的类别为 $\{1, 2, \cdots, C\}$，常用的方式有以下三种：

（1）“一对其余”方式：把多分类问题转换为 C 个“一对其余”的二分类问题. 这种方式共需要 C 个判别函数，其中第 c 个判别函数 f_c 是将类别 c 的样本和不属于类别 c 的样本分开.

（2）“一对一”方式：把多分类问题转换为 $C(C-1)/2$ 个“一对一”的二分类问题. 这种方式共需要 $C(C-1)/2$ 个判别函数，其中第 (i, j) 个判别函数是把类别 i 和类别 j 的样本分开. $1 \leq i < j \leq C$

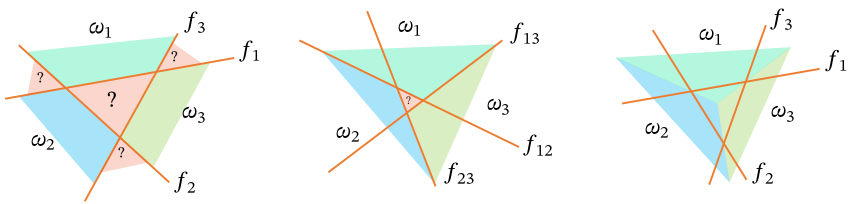
（3）“argmax”方式：这是一种改进的“一对其余”方式，共需要 C 个判别函数

$$f_c(\boldsymbol{x}; \boldsymbol{w}_c) = \boldsymbol{w}_c^{\mathrm{T}} \boldsymbol{x} + b_c, \qquad c \in \{1, \cdots, C\} \tag{3.10}$$

对于样本 $\boldsymbol{x}$，如果存在一个类别 c，相对于所有的其他类别 $\tilde{c}(\tilde{c} \neq c)$ 有 $f_c(\boldsymbol{x}; \boldsymbol{w}_c) > f_{\tilde{c}}(\boldsymbol{x}, \boldsymbol{w}_{\tilde{c}})$，那么 $\boldsymbol{x}$ 属于类别 c. “argmax”方式的预测函数定义为

$$y = \arg\max_{c=1}^{C} f_c(\boldsymbol{x}; \boldsymbol{w}_c). \tag{3.11}$$

“一对其余”方式和“一对一”方式都存在一个缺陷：特征空间中会存在一些难以确定类别的区域，而“argmax”方式很好地解决了这个问题. 图3.3给出了用这三种方式进行多分类的示例，其中红色直线表示判别函数 $f(\cdot) = 0$ 的直线，不同颜色的区域表示预测的三个类别的区域（ω_1、ω_2 和 ω_3）和难以确定类别的区域（‘?’）. 在“argmax”方式中，相邻两类 i 和 j 的决策边界实际上是由 $f_i(\boldsymbol{x}; \boldsymbol{w}_i) - f_j(\boldsymbol{x}; \boldsymbol{w}_j) = 0$ 决定，其法向量为 $\boldsymbol{w}_i - \boldsymbol{w}_j$. 参见习题3-3.



(a) “一对其余”方式 (b) “一对一”方式 (c) “argmax”方式

图 3.3 多分类问题的三种方式

定义 3.2 – 多类线性可分：对于训练集 $\mathcal{D} = \{(\boldsymbol{x}^{(n)}, y^{(n)})\}_{n=1}^{N}$，如果存在 C 个权重向量 $\boldsymbol{w}_1^*, \cdots, \boldsymbol{w}_C^*$，使得第 $c(1 \leq c \leq C)$ 类的所有样本都满足 $f_c(\boldsymbol{x}; \boldsymbol{w}_c^*) > f_{\tilde{c}}(\boldsymbol{x}, \boldsymbol{w}_{\tilde{c}}^*), \forall \tilde{c} \neq c$，那么训练集 $\mathcal{D}$ 是线性可分的.

从上面定义可知，如果数据集是多类线性可分的，那么一定存在一个 “argmax” 方式的线性分类器可以将它们正确分开.

参见习题3-4.

3.2 Logistic 回归

Logistic 回归（Logistic Regression，LR）是一种常用的处理二分类问题的线性模型. 在本节中，我们采用 $y \in \{0, 1\}$ 以符合 Logistic 回归的描述习惯.

为了解决连续的线性函数不适合进行分类的问题，我们引入非线性函数 $g : \mathbb{R}^D \to (0, 1)$ 来预测类别标签的后验概率 $p(y = 1|\boldsymbol{x})$.

$$p(y = 1|\boldsymbol{x}) = g(f(\boldsymbol{x}; \boldsymbol{w})), \tag{3.12}$$

其中 $g(\cdot)$ 通常称为激活函数（Activation Function），其作用是把线性函数的值域从实数区间 “挤压” 到了 $(0, 1)$ 之间，可以用来表示概率. 在统计文献中，$g(\cdot)$ 的逆函数 $g^{-1}(\cdot)$ 也称为联系函数（Link Function）.

在 Logistic 回归中，我们使用 Logistic 函数来作为激活函数. 标签 $y = 1$ 的后验概率为

Logistic 函数参见第B.4.2.1节.

$$p(y = 1|\boldsymbol{x}) = \sigma(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}) \tag{3.13}$$

$$\triangleq \frac{1}{1 + \exp(-\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x})}, \tag{3.14}$$

为简单起见，这里 $\boldsymbol{x} = [x_1, \cdots, x_D, 1]^{\mathrm{T}}$ 和 $\boldsymbol{w} = [w_1, \cdots, w_D, b]^{\mathrm{T}}$ 分别为 $D + 1$ 维的增广特征向量和增广权重向量.

标签 $y = 0$ 的后验概率为

$$p(y = 0|\boldsymbol{x}) = 1 - p(y = 1|\boldsymbol{x}) \tag{3.15}$$

$$= \frac{\exp(-\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x})}{1 + \exp(-\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x})}. \tag{3.16}$$

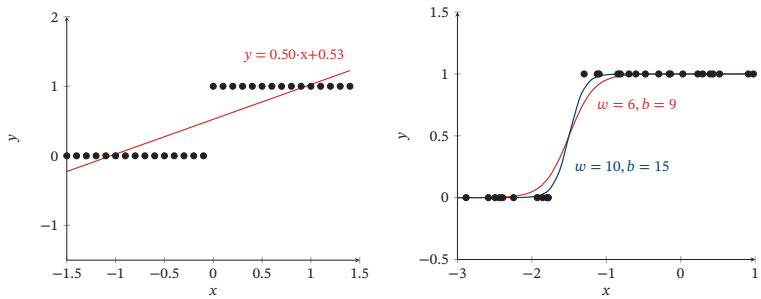
将公式 (3.14) 进行变换后得到

$$\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x} = \log \frac{p(y = 1|\boldsymbol{x})}{1 - p(y = 1|\boldsymbol{x})} \tag{3.17}$$

$$= \log \frac{p(y=1|\boldsymbol{x})}{p(y=0|\boldsymbol{x})}, \tag{3.18}$$

其中 $\frac{p(y=1|\boldsymbol{x})}{p(y=0|\boldsymbol{x})}$ 为样本 $\boldsymbol{x}$ 为正反例后验概率的比值，称为几率（Odds），几率的对数称为对数几率（Log Odds，或 Logit）. 公式(3.17)中等号的左边是线性函数，这样 Logistic 回归可以看作预测值为“标签的对数几率”的线性回归模型. 因此，Logistic 回归也称为对数几率回归（Logit Regression）.

图3.4给出了使用线性回归和 Logistic 回归来解决一维数据的二分类问题的示例.



(a) 线性回归 (b) Logistic 回归

图 3.4 一维数据的二分类问题示例

3.2.1 参数学习

Logistic 回归采用交叉熵作为损失函数，并使用梯度下降法来对参数进行优化.

给定 N 个训练样本 $\{(\boldsymbol{x}^{(n)}, y^{(n)})\}_{n=1}^{N}$，用 Logistic 回归模型对每个样本 $\boldsymbol{x}^{(n)}$ 进行预测，输出其标签为 1 的后验概率，记为 $\hat{y}^{(n)}$，

$$\hat{y}^{(n)} = \sigma(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}^{(n)}), \qquad 1 \leq n \leq N. \tag{3.19}$$

由于 $y^{(n)} \in \{0, 1\}$，样本 $(\boldsymbol{x}^{(n)}, y^{(n)})$ 的真实条件概率可以表示为

$$p_r(y^{(n)}=1|\boldsymbol{x}^{(n)}) = y^{(n)}, \tag{3.20}$$

$$p_r(y^{(n)}=0|\boldsymbol{x}^{(n)}) = 1 - y^{(n)}. \tag{3.21}$$

使用交叉熵损失函数，其风险函数为

$$\mathcal{R}(\boldsymbol{w}) = -\frac{1}{N}\sum_{n=1}^{N}\left(p_r(y^{(n)}=1|\boldsymbol{x}^{(n)})\log\hat{y}^{(n)} + p_r(y^{(n)}=0|\boldsymbol{x}^{(n)})\log(1-\hat{y}^{(n)})\right) \tag{3.22}$$

为简单起见，这里忽略了正则化项.

$$= -\frac{1}{N}\sum_{n=1}^{N}\Big(y^{(n)}\log\hat{y}^{(n)} + (1-y^{(n)})\log(1-\hat{y}^{(n)})\Big). \tag{3.23}$$

风险函数$\mathcal{R}(\boldsymbol{w})$关于参数$\boldsymbol{w}$的偏导数为

$$\frac{\partial\mathcal{R}(\boldsymbol{w})}{\partial\boldsymbol{w}} = -\frac{1}{N}\sum_{n=1}^{N}\left(y^{(n)}\frac{\hat{y}^{(n)}(1-\hat{y}^{(n)})}{\hat{y}^{(n)}}\boldsymbol{x}^{(n)} - (1-y^{(n)})\frac{\hat{y}^{(n)}(1-\hat{y}^{(n)})}{1-\hat{y}^{(n)}}\boldsymbol{x}^{(n)}\right) \tag{3.24}$$

$$= -\frac{1}{N}\sum_{n=1}^{N}\left(y^{(n)}(1-\hat{y}^{(n)})\boldsymbol{x}^{(n)} - (1-y^{(n)})\hat{y}^{(n)}\boldsymbol{x}^{(n)}\right) \tag{3.25}$$

$$= -\frac{1}{N}\sum_{n=1}^{N}\boldsymbol{x}^{(n)}\big(y^{(n)} - \hat{y}^{(n)}\big). \tag{3.26}$$

$\hat{y}$为Logistic函数，因此有$\frac{\partial\hat{y}}{\partial\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}} = \hat{y}^{(n)}(1-\hat{y}^{(n)})$. 参见第B.4.2.1节.

采用梯度下降法，Logistic回归的训练过程为：初始化$\boldsymbol{w}_0 \leftarrow 0$，然后通过下式来迭代更新参数：

$$\boldsymbol{w}_{t+1} \leftarrow \boldsymbol{w}_t + \alpha\frac{1}{N}\sum_{n=1}^{N}\boldsymbol{x}^{(n)}\Big(y^{(n)} - \hat{y}_{\boldsymbol{w}_t}^{(n)}\Big), \tag{3.27}$$

其中α是学习率，$\hat{y}_{\boldsymbol{w}_t}^{(n)}$是当参数为$\boldsymbol{w}_t$时，Logistic回归模型的输出.

从公式(3.23)可知，风险函数$\mathcal{R}(\boldsymbol{w})$是关于参数$\boldsymbol{w}$的连续可导的凸函数. 因此除了梯度下降法之外，Logistic回归还可以用高阶的优化方法（比如牛顿法）来进行优化.

3.3 Softmax回归

Softmax回归（Softmax Regression），也称为多项（Multinomial）或多类（Multi-Class）的Logistic回归，是Logistic回归在多分类问题上的推广.

Softmax回归也可以看作一种条件最大熵模型，参见第11.1.5.1节.

对于多类问题，类别标签$y \in \{1, 2, \cdots, C\}$可以有$C$个取值. 给定一个样本$\boldsymbol{x}$，Softmax回归预测的属于类别$c$的条件概率为

$$p(y=c|\boldsymbol{x}) = \text{softmax}(\boldsymbol{w}_c^{\mathrm{T}}\boldsymbol{x}) \tag{3.28}$$

$$= \frac{\exp(\boldsymbol{w}_c^{\mathrm{T}}\boldsymbol{x})}{\sum_{c'=1}^{C}\exp(\boldsymbol{w}_{c'}^{\mathrm{T}}\boldsymbol{x})}, \tag{3.29}$$

Softmax函数参见第B.4.2.2节.

其中$\boldsymbol{w}_c$是第c类的权重向量.

Softmax回归的决策函数可以表示为

$$\hat{y} = \arg\max_{c=1}^{C} p(y=c|\boldsymbol{x}) \tag{3.30}$$

$$= \arg\max_{c=1}^{C} \boldsymbol{w}_c^{\mathrm{T}}\boldsymbol{x}. \tag{3.31}$$

与Logistic回归的关系 当类别数 $C=2$ 时，Softmax回归的决策函数为

$$\hat{y} = \arg\max_{y\in\{0,1\}} \boldsymbol{w}_y^{\mathrm{T}}\boldsymbol{x} \tag{3.32}$$

$$= I\Big(\boldsymbol{w}_1^{\mathrm{T}}\boldsymbol{x} - \boldsymbol{w}_0^{\mathrm{T}}\boldsymbol{x} > 0\Big) \tag{3.33}$$

$$= I\Big((\boldsymbol{w}_1 - \boldsymbol{w}_0)^{\mathrm{T}}\boldsymbol{x} > 0\Big), \tag{3.34}$$

其中 $I(\cdot)$ 是指示函数. 对比公式(3.5)中的二分类决策函数，可以发现二分类中的权重向量 $\boldsymbol{w} = \boldsymbol{w}_1 - \boldsymbol{w}_0$.

向量表示 公式(3.29)用向量形式可以写为

$$\hat{\boldsymbol{y}} = \mathrm{softmax}(\boldsymbol{W}^{\mathrm{T}}\boldsymbol{x}) \tag{3.35}$$

$$= \frac{\exp(\boldsymbol{W}^{\mathrm{T}}\boldsymbol{x})}{\mathbf{1}_C^{\mathrm{T}}\exp(\boldsymbol{W}^{\mathrm{T}}\boldsymbol{x})}, \tag{3.36}$$

其中 $\boldsymbol{W} = [\boldsymbol{w}_1, \cdots, \boldsymbol{w}_C]$ 是由 C 个类的权重向量组成的矩阵，$\mathbf{1}_C$ 为 C 维的全1向量，$\hat{\boldsymbol{y}} \in \mathbb{R}^C$ 为所有类别的预测条件概率组成的向量，第 c 维的值是第 c 类的预测条件概率.

3.3.1 参数学习

给定 N 个训练样本 $\{(\boldsymbol{x}^{(n)}, y^{(n)})\}_{n=1}^N$，Softmax回归使用交叉熵损失函数来学习最优的参数矩阵 $\boldsymbol{W}$.

为了方便起见，我们用 C 维的one-hot向量 $\boldsymbol{y} \in \{0,1\}^C$ 来表示类别标签. 对于类别 c，其向量表示为

$$\boldsymbol{y} = [I(1=c), I(2=c), \cdots, I(C=c)]^{\mathrm{T}}, \tag{3.37}$$

其中 $I(\cdot)$ 是指示函数.

采用交叉熵损失函数，Softmax回归模型的风险函数为

为简单起见，这里忽略了正则化项.

$$\mathcal{R}(\boldsymbol{W}) = -\frac{1}{N}\sum_{n=1}^{N}\sum_{c=1}^{C}\boldsymbol{y}_c^{(n)}\log\hat{\boldsymbol{y}}_c^{(n)} \tag{3.38}$$

$$= -\frac{1}{N}\sum_{n=1}^{N}(\boldsymbol{y}^{(n)})^{\mathrm{T}}\log\hat{\boldsymbol{y}}^{(n)}, \tag{3.39}$$

其中 $\hat{\boldsymbol{y}}^{(n)} = \mathrm{softmax}(\boldsymbol{W}^{\mathrm{T}}\boldsymbol{x}^{(n)})$ 为样本 $\boldsymbol{x}^{(n)}$ 在每个类别的后验概率.

风险函数 $\mathcal{R}(\boldsymbol{W})$ 关于 $\boldsymbol{W}$ 的梯度为

$$\frac{\partial\mathcal{R}(\boldsymbol{W})}{\partial\boldsymbol{W}} = -\frac{1}{N}\sum_{n=1}^{N}\boldsymbol{x}^{(n)}\big(\boldsymbol{y}^{(n)} - \hat{\boldsymbol{y}}^{(n)}\big)^{\mathrm{T}}. \tag{3.40}$$

证明. 计算公式(3.40)中的梯度，关键在于计算每个样本的损失函数 $\mathcal{L}^{(n)}(\boldsymbol{W}) = -(\boldsymbol{y}^{(n)})^{\mathrm{T}} \log \hat{\boldsymbol{y}}^{(n)}$ 关于参数 $\boldsymbol{W}$ 的梯度，其中需要用到的两个导数公式为

（1） 若 $\boldsymbol{y} = \mathrm{softmax}(\boldsymbol{z})$，则 $\frac{\partial \boldsymbol{y}}{\partial \boldsymbol{z}} = \mathrm{diag}(\boldsymbol{y}) - \boldsymbol{y}\boldsymbol{y}^{\mathrm{T}}$.

Softmax函数的导数参见第B.4.2.2节.

（2） 若 $\boldsymbol{z} = \boldsymbol{W}^{\mathrm{T}}\boldsymbol{x} = [\boldsymbol{w}_1^{\mathrm{T}}\boldsymbol{x}, \boldsymbol{w}_2^{\mathrm{T}}\boldsymbol{x}, \cdots, \boldsymbol{w}_C^{\mathrm{T}}\boldsymbol{x}]^{\mathrm{T}}$，则 $\frac{\partial \boldsymbol{z}}{\partial \boldsymbol{w}_c}$ 为第 c 列为 $\boldsymbol{x}$，其余为0的矩阵.

$$\frac{\partial \boldsymbol{z}}{\partial \boldsymbol{w}_c} = [\frac{\partial \boldsymbol{w}_1^{\mathrm{T}}\boldsymbol{x}}{\partial \boldsymbol{w}_c}, \frac{\partial \boldsymbol{w}_2^{\mathrm{T}}\boldsymbol{x}}{\partial \boldsymbol{w}_c}, \cdots, \frac{\partial \boldsymbol{w}_C^{\mathrm{T}}\boldsymbol{x}}{\partial \boldsymbol{w}_c}] \tag{3.41}$$

$$= [\boldsymbol{0}, \boldsymbol{0}, \cdots, \boldsymbol{x}, \cdots, \boldsymbol{0}] \tag{3.42}$$

$$\triangleq \mathbb{M}_c(\boldsymbol{x}). \tag{3.43}$$

根据链式法则，$\mathcal{L}^{(n)}(\boldsymbol{W}) = -(\boldsymbol{y}^{(n)})^{\mathrm{T}} \log \hat{\boldsymbol{y}}^{(n)}$ 关于 $\boldsymbol{w}_c$ 的偏导数为

$$\frac{\partial \mathcal{L}^{(n)}(\boldsymbol{W})}{\partial \boldsymbol{w}_c} = -\frac{\partial \left((\boldsymbol{y}^{(n)})^{\mathrm{T}} \log \hat{\boldsymbol{y}}^{(n)}\right)}{\partial \boldsymbol{w}_c} \tag{3.44}$$

$$= -\frac{\partial \boldsymbol{z}^{(n)}}{\partial \boldsymbol{w}_c} \frac{\partial \hat{\boldsymbol{y}}^{(n)}}{\partial \boldsymbol{z}^{(n)}} \frac{\partial \log \hat{\boldsymbol{y}}^{(n)}}{\partial \hat{\boldsymbol{y}}^{(n)}} \boldsymbol{y}^{(n)} \tag{3.45}$$

$$= -\mathbb{M}_c(\boldsymbol{x}^{(n)}) \left(\mathrm{diag}\left(\hat{\boldsymbol{y}}^{(n)}\right) - \hat{\boldsymbol{y}}^{(n)}(\hat{\boldsymbol{y}}^{(n)})^{\mathrm{T}}\right) \left(\mathrm{diag}(\hat{\boldsymbol{y}}^{(n)})\right)^{-1} \boldsymbol{y}^{(n)} \tag{3.46}$$

$\boldsymbol{y}^{\mathrm{T}} \mathrm{diag}(\boldsymbol{y})^{-1} = \boldsymbol{1}_C^{\mathrm{T}}$ 为全1的行向量.

$$= -\mathbb{M}_c(\boldsymbol{x}^{(n)}) \left(\boldsymbol{I} - \hat{\boldsymbol{y}}^{(n)} \boldsymbol{1}_C^{\mathrm{T}}\right) \boldsymbol{y}^{(n)} \tag{3.47}$$

$$= -\mathbb{M}_c(\boldsymbol{x}^{(n)}) \left(\boldsymbol{y}^{(n)} - \hat{\boldsymbol{y}}^{(n)} \boldsymbol{1}_C^{\mathrm{T}} \boldsymbol{y}^{(n)}\right) \tag{3.48}$$

$\boldsymbol{y}$ 为one-hot向量，所以 $\boldsymbol{1}_C^{\mathrm{T}} \boldsymbol{y} = 1$.

$$= -\mathbb{M}_c(\boldsymbol{x}^{(n)}) \left(\boldsymbol{y}^{(n)} - \hat{\boldsymbol{y}}^{(n)}\right) \tag{3.49}$$

$$= -\boldsymbol{x}^{(n)} \left[\boldsymbol{y}^{(n)} - \hat{\boldsymbol{y}}^{(n)}\right]_c. \tag{3.50}$$

公式(3.50)也可以表示为非向量形式，即

$$\frac{\partial \mathcal{L}^{(n)}(\boldsymbol{W})}{\partial \boldsymbol{w}_c} = -\boldsymbol{x}^{(n)} \left(I(y^{(n)} = c) - \hat{\boldsymbol{y}}_c^{(n)}\right), \tag{3.51}$$

其中 $I(\cdot)$ 是指示函数.

根据公式(3.50)可以得到

$$\frac{\partial \mathcal{L}^{(n)}(\boldsymbol{W})}{\partial \boldsymbol{W}} = -\boldsymbol{x}^{(n)} \left(\boldsymbol{y}^{(n)} - \hat{\boldsymbol{y}}^{(n)}\right)^{\mathrm{T}}. \tag{3.52}$$

□

采用梯度下降法，Softmax回归的训练过程为：初始化 $\boldsymbol{W}_0 \leftarrow 0$，然后通过下式进行迭代更新：

$$\boldsymbol{W}_{t+1} \leftarrow \boldsymbol{W}_t + \alpha \left(\frac{1}{N} \sum_{n=1}^{N} \boldsymbol{x}^{(n)} \left(\boldsymbol{y}^{(n)} - \hat{\boldsymbol{y}}_{\boldsymbol{W}_t}^{(n)}\right)^{\mathrm{T}}\right), \tag{3.53}$$

其中 α 是学习率，$\hat{\boldsymbol{y}}_{\boldsymbol{W}_t}^{(n)}$ 是当参数为 $\boldsymbol{W}_t$ 时，Softmax回归模型的输出.

要注意的是，Softmax 回归中使用的 C 个权重向量是冗余的，即对所有的权重向量都减去一个同样的向量 $\boldsymbol{v}$，不改变其输出结果. 因此，Softmax 回归往往需要使用正则化来约束其参数. 此外，我们还可以利用这个特性来避免计算 Softmax 函数时在数值计算上溢出问题.

3.4 感知器

感知器（Perceptron）由 Frank Roseblatt 于 1957 年提出，是一种广泛使用的线性分类器. 感知器可谓是最简单的人工神经网络，只有一个神经元.

Frank Rosenblatt（1928~1971），美国心理学家，人工智能领域开拓者.
最早发明的感知器是一台机器而不是一种算法，后来才被实现为 IBM 704 机器上可运行的程序.

感知器是对生物神经元的简单数学模拟，有与生物神经元相对应的部件，如权重（突触）、偏置（阈值）及激活函数（细胞体），输出为 $+1$ 或 -1.

感知器是一种简单的两类线性分类模型，其分类准则与公式 (3.5) 相同，即

$$\hat{y} = \text{sgn}(\boldsymbol{w}^{\top}\boldsymbol{x}). \tag{3.54}$$

3.4.1 参数学习

感知器学习算法也是一个经典的线性分类器的参数学习算法.

给定 N 个样本的训练集：$\{(\boldsymbol{x}^{(n)}, y^{(n)})\}_{n=1}^{N}$，其中 $y^{(n)} \in \{+1, -1\}$，感知器学习算法试图找到一组参数 $\boldsymbol{w}^*$，使得对于每个样本 $(\boldsymbol{x}^{(n)}, y^{(n)})$ 有

$$y^{(n)}\boldsymbol{w}^{*\top}\boldsymbol{x}^{(n)} > 0, \qquad \forall n \in \{1, \cdots, N\}. \tag{3.55}$$

感知器的学习算法是一种错误驱动的在线学习算法 [Rosenblatt, 1958]. 先初始化一个权重向量 $\boldsymbol{w} \leftarrow 0$（通常是全零向量），然后每次分错一个样本 $(\boldsymbol{x}, y)$ 时，即 $y\boldsymbol{w}^{\top}\boldsymbol{x} < 0$，就用这个样本来更新权重.

$$\boldsymbol{w} \leftarrow \boldsymbol{w} + y\boldsymbol{x}. \tag{3.56}$$

具体的感知器参数学习策略如算法3.1所示.

根据感知器的学习策略，可以反推出感知器的损失函数为

$$\mathcal{L}(\boldsymbol{w}; \boldsymbol{x}, y) = \max(0, -y\boldsymbol{w}^{\top}\boldsymbol{x}). \tag{3.57}$$

采用随机梯度下降，其每次更新的梯度为

$$\frac{\partial \mathcal{L}(\boldsymbol{w}; \boldsymbol{x}, y)}{\partial \boldsymbol{w}} = \begin{cases} 0 & \text{if} \quad y\boldsymbol{w}^{\top}\boldsymbol{x} > 0, \\ -y\boldsymbol{x} & \text{if} \quad y\boldsymbol{w}^{\top}\boldsymbol{x} < 0. \end{cases} \tag{3.58}$$

算法 3.1 两类感知器的参数学习算法

输入: 训练集 $\mathcal{D} = \{(\boldsymbol{x}^{(n)}, y^{(n)})\}_{n=1}^{N}$，最大迭代次数 T

初始化：$\boldsymbol{w}_0 \leftarrow 0, k \leftarrow 0, t \leftarrow 0$；
repeat
  对训练集 $\mathcal{D}$ 中的样本随机排序；
  for $n = 1 \cdots N$ **do**
    选取一个样本 $(\boldsymbol{x}^{(n)}, y^{(n)})$；
    if $\boldsymbol{w}_k^{\mathrm{T}}(y^{(n)}\boldsymbol{x}^{(n)}) \leq 0$ **then**
      $\boldsymbol{w}_{k+1} \leftarrow \boldsymbol{w}_k + y^{(n)}\boldsymbol{x}^{(n)}$；
      $k \leftarrow k + 1$；
    end
    $t \leftarrow t + 1$；
    if $t = T$ **then** break； // 达到最大迭代次数
  end
until $t = T$；
输出: $\boldsymbol{w}_k$

图3.5给出了感知器参数学习的更新过程，其中红色实心点为正例，蓝色空心点为负例. 黑色箭头表示当前的权重向量，红色虚线箭头表示权重的更新方向.

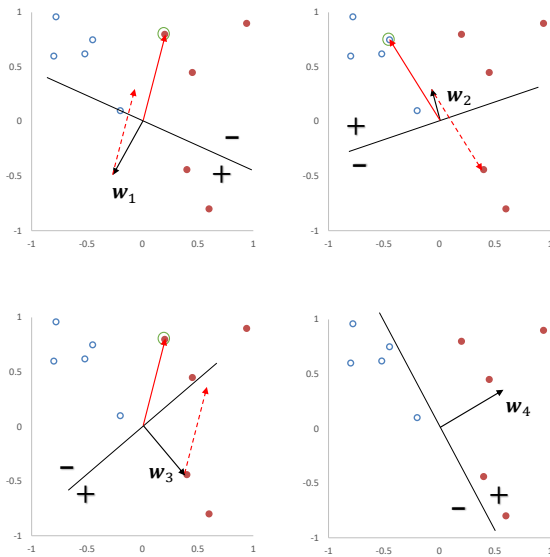


图 3.5 感知器参数学习的更新过程

3.4.2 感知器的收敛性

[Novikoff, 1963] 证明对于两类问题，如果训练集是线性可分的，那么感知器算法可以在有限次迭代后收敛. 然而，如果训练集不是线性可分的，那么这个算法则不能确保会收敛.

当数据集是两类线性可分时，对于训练集 $\mathcal{D} = \{(\boldsymbol{x}^{(n)}, y^{(n)})\}_{n=1}^{N}$，其中 $\boldsymbol{x}^{(n)}$ 为样本的增广特征向量，$y^{(n)} \in \{-1, 1\}$，那么存在一个正的常数 $\gamma(\gamma > 0)$ 和权重向量 $\boldsymbol{w}^*$，并且 $\|\boldsymbol{w}^*\| = 1$，对所有 n 都满足 $(\boldsymbol{w}^*)^{\mathrm{T}}(y^{(n)}\boldsymbol{x}^{(n)}) \geq \gamma$. 我们可以证明如下定理.

参见定义3.1.

定理 3.1 – 感知器收敛性：给定训练集 $\mathcal{D} = \{(\boldsymbol{x}^{(n)}, y^{(n)})\}_{n=1}^{N}$，令 R 是训练集中最大的特征向量的模，即

$$R = \max_n \|x^{(n)}\|.$$

如果训练集 $\mathcal{D}$ 线性可分，两类感知器的参数学习算法3.1的权重更新次数不超过 $\frac{R^2}{\gamma^2}$.

证明. 感知器的权重向量的更新方式为

$$\boldsymbol{w}_k = \boldsymbol{w}_{k-1} + y^{(k)}\boldsymbol{x}^{(k)}, \tag{3.59}$$

其中 $\boldsymbol{x}^{(k)}, y^{(k)}$ 表示第 k 个错误分类的样本.

因为初始权重向量为 0，在第 K 次更新时感知器的权重向量为

$$\boldsymbol{w}_K = \sum_{k=1}^{K} y^{(k)}\boldsymbol{x}^{(k)}. \tag{3.60}$$

分别计算 $\|\boldsymbol{w}_K\|^2$ 的上下界：

（1）$\|\boldsymbol{w}_K\|^2$ 的上界为

$$\begin{aligned}
\|\boldsymbol{w}_K\|^2 &= \|\boldsymbol{w}_{K-1} + y^{(K)}\boldsymbol{x}^{(K)}\|^2 && (3.61)\\
&= \|\boldsymbol{w}_{K-1}\|^2 + \|y^{(K)}\boldsymbol{x}^{(K)}\|^2 + 2y^{(K)}\boldsymbol{w}_{K-1}^{\mathrm{T}}\boldsymbol{x}^{(K)} && (3.62)\\
&\leq \|\boldsymbol{w}_{K-1}\|^2 + R^2 && (3.63)\\
&\leq \|\boldsymbol{w}_{K-2}\|^2 + 2R^2 && (3.64)\\
&\leq KR^2. && (3.65)
\end{aligned}$$

$y_k\boldsymbol{w}_{K-1}^{\mathrm{T}}\boldsymbol{x}^{(K)} \leq 0.$

（2）$\|\boldsymbol{w}_K\|^2$ 的下界为

$$\|\boldsymbol{w}_K\|^2 = \|\boldsymbol{w}^*\|^2 \cdot \|\boldsymbol{w}_K\|^2 \tag{3.66}$$

$\|\boldsymbol{w}^*\| = 1$. 两个向量内积的平方一定小于等于这两个向量的模的乘积.

$$\geq \|\boldsymbol{w}^{*\mathrm{T}}\boldsymbol{w}_K\|^2 \tag{3.67}$$

$$= \|\boldsymbol{w}^{*\mathrm{T}} \sum_{k=1}^{K} (y^{(k)}\boldsymbol{x}^{(k)})\|^2 \tag{3.68}$$

$$= \|\sum_{k=1}^{K} \boldsymbol{w}^{*\mathrm{T}}(y^{(k)}\boldsymbol{x}^{(k)})\|^2 \tag{3.69}$$

$\boldsymbol{w}^{*\mathrm{T}}(y^{(n)}\boldsymbol{x}^{(n)}) \geq \gamma, \forall n$.

$$\geq K^2\gamma^2. \tag{3.70}$$

由公式(3.65)和公式(3.70),得到

$$K^2\gamma^2 \leq \|\boldsymbol{w}_K\|^2 \leq KR^2. \tag{3.71}$$

取最左和最右的两项,进一步得到,$K^2\gamma^2 \leq KR^2$. 然后两边都除 K,最终得到 $K \leq \frac{R^2}{\gamma^2}$. 因此,在线性可分的条件下,算法3.1会在 $\frac{R^2}{\gamma^2}$ 步内收敛. □

虽然感知器在线性可分的数据上可以保证收敛,但其存在以下不足:

（1） 在数据集线性可分时,感知器虽然可以找到一个超平面把两类数据分开,但并不能保证其泛化能力.

（2） 感知器对样本顺序比较敏感. 每次迭代的顺序不一致时,找到的分割超平面也往往不一致.

（3） 如果训练集不是线性可分的,就永远不会收敛.

3.4.3 参数平均感知器

根据定理3.1, 如果训练数据是线性可分的, 那么感知器可以找到一个判别函数来分割不同类的数据. 如果间隔 γ 越大,收敛越快. 但是感知器并不能保证找到的判别函数是最优的（比如泛化能力高）,这样可能导致过拟合.

感知器学习到的权重向量和训练样本的顺序相关. 在迭代次序上排在后面的错误样本比前面的错误样本, 对最终的权重向量影响更大. 比如有 1 000 个训练样本, 在迭代 100 个样本后, 感知器已经学习到一个很好的权重向量. 在接下来的 899 个样本上都预测正确, 也没有更新权重向量. 但是, 在最后第 1 000 个样本时预测错误,并更新了权重. 这次更新可能反而使得权重向量变差.

为了提高感知器的鲁棒性和泛化能力, 我们可以将在感知器学习过程中的所有 K 个权重向量保存起来, 并赋予每个权重向量 $\boldsymbol{w}_k$ 一个置信系数 c_k（$1 \leq$

K 为感知器在训练中权重向量的总更新次数.

$k \leq K$）. 最终的分类结果通过这 K 个不同权重的感知器投票决定，这个模型也称为投票感知器（Voted Perceptron）[Freund et al., 1999].

令 τ_k 为第 k 次更新权重 $\boldsymbol{w}_k$ 时的迭代次数（即训练过的样本数量），τ_{k+1} 为下次权重更新时的迭代次数，则权重 $\boldsymbol{w}_k$ 的置信系数 c_k 设置为从 τ_k 到 τ_{k+1} 之间间隔的迭代次数，即 $c_k = \tau_{k+1} - \tau_k$. 置信系数 c_k 越大，说明权重 $\boldsymbol{w}_k$ 在之后的训练过程中正确分类样本的数量越多，越值得信赖.

这样，投票感知器的形式为

$$\hat{y} = \text{sgn}\Big(\sum_{k=1}^{K} c_k \,\text{sgn}(\boldsymbol{w}_k^{\mathrm{T}}\boldsymbol{x})\Big), \tag{3.72}$$

投票感知器是一种集成模型，参见第10.1节.

其中 $\text{sgn}(\cdot)$ 为符号函数.

投票感知器虽然提高了感知器的泛化能力，但是需要保存 K 个权重向量. 在实际操作中会带来额外的开销. 因此，人们经常会使用一个简化的版本，通过使用“参数平均”的策略来减少投票感知器的参数数量，也叫作平均感知器（Averaged Perceptron）[Collins, 2002]. 平均感知器的形式为

$$\hat{y} = \text{sgn}\Big(\frac{1}{T}\sum_{k=1}^{K} c_k(\boldsymbol{w}_k^{\mathrm{T}}\boldsymbol{x})\Big) \tag{3.73}$$

$$= \text{sgn}\Big(\frac{1}{T}(\sum_{k=1}^{K} c_k\boldsymbol{w}_k)^{\mathrm{T}}\boldsymbol{x}\Big) \tag{3.74}$$

$$= \text{sgn}\Big((\frac{1}{T}\sum_{t=1}^{T} \boldsymbol{w}_t)^{\mathrm{T}}\boldsymbol{x}\Big) \tag{3.75}$$

$$= \text{sgn}(\bar{\boldsymbol{w}}^{\mathrm{T}}\boldsymbol{x}), \tag{3.76}$$

其中 T 为迭代总回合数，$\bar{\boldsymbol{w}}$ 为 T 次迭代的平均权重向量. 这个方法非常简单，只需要在算法3.1中增加一个 $\bar{\boldsymbol{w}}$，并且在每次迭代时都更新 $\bar{\boldsymbol{w}}$：

$$\bar{\boldsymbol{w}} \leftarrow \bar{\boldsymbol{w}} + \boldsymbol{w}_t. \tag{3.77}$$

但这个方法需要在处理每一个样本时都要更新 $\bar{\boldsymbol{w}}$. 因为 $\bar{\boldsymbol{w}}$ 和 $\boldsymbol{w}_t$ 都是稠密向量，所以更新操作比较费时. 为了提高迭代速度，有很多改进的方法，让这个更新只需要在错误预测发生时才进行更新.

算法3.2给出了一个改进的平均感知器算法的训练过程 [Daumé III, 2012].

参见习题3-7.

算法 3.2 一种改进的平均感知器参数学习算法

输入: 训练集 $\{(\boldsymbol{x}^{(n)}, y^{(n)})\}_{n=1}^{N}$,最大迭代次数 T

初始化:$\boldsymbol{w} \leftarrow 0, \boldsymbol{u} \leftarrow 0, t \leftarrow 0$;
repeat
  对训练集 $\mathcal{D}$ 中的样本随机排序;
  for $n = 1 \cdots N$ **do**
    选取一个样本 $(\boldsymbol{x}^{(n)}, y^{(n)})$;
    计算预测类别 $\hat{y}_t$;
    if $\hat{y}_t \neq y_t$ **then**
      $\boldsymbol{w} \leftarrow \boldsymbol{w} + y^{(n)}\boldsymbol{x}^{(n)}$;
      $\boldsymbol{u} \leftarrow \boldsymbol{u} + ty^{(n)}\boldsymbol{x}^{(n)}$;
    end
    $t \leftarrow t + 1$;
    if $t = T$ **then** break; // 达到最大迭代次数
  end
until $t = T$;
$\bar{\boldsymbol{w}} = \boldsymbol{w}_T - \frac{1}{T}\boldsymbol{u}$;
输出: $\bar{\boldsymbol{w}}$

3.4.4 扩展到多分类

原始的感知器是一种二分类模型,但也可以很容易地扩展到多分类问题,甚至是更一般的结构化学习问题 [Collins, 2002].

之前介绍的分类模型中,分类函数都是在输入 $\boldsymbol{x}$ 的特征空间上. 为了使得感知器可以处理更复杂的输出,我们引入一个构建在输入输出联合空间上的特征函数 $\boldsymbol{\phi}(\boldsymbol{x}, \boldsymbol{y})$,将样本对 $(\boldsymbol{x}, \boldsymbol{y})$ 映射到一个特征向量空间.

通过引入特征函数 $\boldsymbol{\phi}(\boldsymbol{x}, \boldsymbol{y})$,感知器不但可以用于多分类问题,也可以用于结构化学习问题,比如输出是序列形式.

在联合特征空间中,我们可以建立一个广义的感知器模型,

$$\hat{\boldsymbol{y}} = \underset{\boldsymbol{y} \in \mathtt{Gen}(\boldsymbol{x})}{\arg\max}\, \boldsymbol{w}^{\mathrm{T}} \boldsymbol{\phi}(\boldsymbol{x}, \boldsymbol{y}), \tag{3.78}$$

其中 $\boldsymbol{w}$ 为权重向量,$\mathtt{Gen}(\boldsymbol{x})$ 表示输入 $\boldsymbol{x}$ 所有的输出目标集合.

广义感知器模型一般用来处理结构化学习问题. 当用广义感知器模型来处理 C 分类问题时,$\boldsymbol{y} \in \{0, 1\}^C$ 为类别的 one-hot 向量表示. 在 C 分类问题中,一种常用的特征函数 $\boldsymbol{\phi}(\boldsymbol{x}, \boldsymbol{y})$ 是 $\boldsymbol{x}$ 和 $\boldsymbol{y}$ 的外积,即

外积的定义参见公式(A.28).

$$\boldsymbol{\phi}(\boldsymbol{x}, \boldsymbol{y}) = \mathrm{vec}(\boldsymbol{x}\boldsymbol{y}^{\mathrm{T}}) \in \mathbb{R}^{(D \times C)}, \tag{3.79}$$

其中 $\mathrm{vec}(\cdot)$ 是向量化算子,$\boldsymbol{\phi}(\boldsymbol{x}, \boldsymbol{y})$ 为 $(D \times C)$ 维的向量.

给定样本 $(\boldsymbol{x}, y)$，若 $\boldsymbol{x} \in \mathbb{R}^D$，$y$ 为第 c 维为 1 的 one-hot 向量，则

$$\boldsymbol{\phi}(\boldsymbol{x}, y) = \begin{bmatrix} \vdots \\ 0 \\ x_1 \\ \vdots \\ x_D \\ 0 \\ \vdots \end{bmatrix} \begin{matrix} \\ \\ \leftarrow \text{第}(c-1)\times D+1\text{行} \\ \\ \leftarrow \text{第}(c-1)\times D+D\text{行} \\ \\ \end{matrix} . \tag{3.80}$$

广义感知器算法的训练过程如算法3.3所示.

算法 3.3 广义感知器参数学习算法

输入: 训练集:$\{(\boldsymbol{x}^{(n)}, y^{(n)})\}_{n=1}^{N}$，最大迭代次数 T

初始化：$\boldsymbol{w}_0 \leftarrow 0$, $k \leftarrow 0$, $t \leftarrow 0$；
repeat
  对训练集 $\mathcal{D}$ 中的样本随机排序;
  for $n = 1 \cdots N$ **do**
    选取一个样本 $(\boldsymbol{x}^{(n)}, y^{(n)})$;
    用公式(3.78)计算预测类别 $\hat{y}^{(n)}$;
    if $\hat{y}^{(n)} \neq y^{(n)}$ **then**
      $\boldsymbol{w}_{k+1} \leftarrow \boldsymbol{w}_k + \left(\boldsymbol{\phi}(\boldsymbol{x}^{(n)}, y^{(n)}) - \boldsymbol{\phi}(\boldsymbol{x}^{(n)}, \hat{y}^{(n)})\right)$;
      $k = k + 1$；
    end
    $t = t + 1$；
    if $t = T$ **then** break; // 达到最大迭代次数
  end
until $t = T$;
输出: $\boldsymbol{w}_k$

3.4.4.1 广义感知器的收敛性

广义感知器在满足广义线性可分条件时，也能够保证在有限步骤内收敛. 广义线性可分条件的定义如下：

定义 3.3–广义线性可分：对于训练集 $\mathcal{D} = \{(\boldsymbol{x}^{(n)}, \boldsymbol{y}^{(n)})\}_{n=1}^{N}$，如果存在一个正的常数 $\gamma(\gamma > 0)$ 和权重向量 $\boldsymbol{w}^*$，并且 $\|\boldsymbol{w}^*\| = 1$，对所有 n 都满足 $\langle \boldsymbol{w}^*, \phi(\boldsymbol{x}^{(n)}, \boldsymbol{y}^{(n)})\rangle - \langle \boldsymbol{w}^*, \phi(\boldsymbol{x}^{(n)}, \boldsymbol{y})\rangle \geq \gamma, \boldsymbol{y} \neq \boldsymbol{y}^{(n)}$（$\phi(\boldsymbol{x}^{(n)}, \boldsymbol{y}^{(n)}) \in \mathbb{R}^D$ 为样本 $\boldsymbol{x}^{(n)}, \boldsymbol{y}^{(n)}$ 的联合特征向量），那么训练集 $\mathcal{D}$ 在联合特征向量空间中是线性可分的.

广义线性可分是多类线性可分的扩展，参见定义3.2.

广义感知器的收敛性定义如下：

定理 3.2–广义感知器收敛性：如果训练集 $\mathcal{D} = \{(\boldsymbol{x}^{(n)}, \boldsymbol{y}^{(n)})\}_{n=1}^{N}$ 是广义线性可分的，并令 R 是所有样本中真实标签和错误标签在特征空间 $\phi(\boldsymbol{x}, \boldsymbol{y})$ 最远的距离，即

$$R = \max_{n} \max_{\boldsymbol{z} \neq \boldsymbol{y}^{(n)}} \|\phi(\boldsymbol{x}^{(n)}, \boldsymbol{y}^{(n)}) - \phi(\boldsymbol{x}^{(n)}, \boldsymbol{z})\|, \tag{3.81}$$

那么广义感知器参数学习算法3.3的权重更新次数不超过 $\frac{R^2}{\gamma^2}$.

[Collins, 2002]给出了广义感知器在广义线性可分的收敛性证明，具体推导过程和两类感知器比较类似.

参见习题3-8.

3.5 支持向量机

支持向量机（Support Vector Machine，SVM）是一个经典的二分类算法，其找到的分割超平面具有更好的鲁棒性，因此广泛使用在很多任务上，并表现出了很强优势.

给定一个二分类器数据集 $\mathcal{D} = \{(\boldsymbol{x}^{(n)}, y^{(n)})\}_{n=1}^{N}$，其中 $y_n \in \{+1, -1\}$，如果两类样本是线性可分的，即存在一个超平面

本节中不使用增广的特征向量和特征权重.

$$\boldsymbol{w}^{\top}\boldsymbol{x} + b = 0 \tag{3.82}$$

将两类样本分开，那么对于每个样本都有 $y^{(n)}(\boldsymbol{w}^{\top}\boldsymbol{x}^{(n)} + b) > 0$.

数据集 $\mathcal{D}$ 中每个样本 $\boldsymbol{x}^{(n)}$ 到分割超平面的距离为：

$$\gamma^{(n)} = \frac{|\boldsymbol{w}^{\top}\boldsymbol{x}^{(n)} + b|}{\|\boldsymbol{w}\|} = \frac{y^{(n)}(\boldsymbol{w}^{\top}\boldsymbol{x}^{(n)} + b)}{\|\boldsymbol{w}\|}. \tag{3.83}$$

我们定义间隔（Margin）γ 为整个数据集 D 中所有样本到分割超平面的最短距离：

$$\gamma = \min_n \gamma^{(n)}. \tag{3.84}$$

如果间隔 γ 越大，其分割超平面对两个数据集的划分越稳定，不容易受噪声等因素影响. 支持向量机的目标是寻找一个超平面 $(\boldsymbol{w}^*, b^*)$ 使得 γ 最大，即

$$\begin{aligned} \max_{\boldsymbol{w},b} \quad & \gamma \\ \text{s.t.} \quad & \frac{y^{(n)}(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}^{(n)} + b)}{\|\boldsymbol{w}\|} \geq \gamma, \forall n \in \{1, \cdots, N\}. \end{aligned} \tag{3.85}$$

由于同时缩放 $\boldsymbol{w} \to k\boldsymbol{w}$ 和 $b \to kb$ 不会改变样本 $\boldsymbol{x}^{(n)}$ 到分割超平面的距离，我们可以限制 $\|\boldsymbol{w}\| \cdot \gamma = 1$，则公式 (3.85) 等价于

间隔为 $\gamma = \frac{1}{\|\boldsymbol{w}\|}$.

$$\begin{aligned} \max_{\boldsymbol{w},b} \quad & \frac{1}{\|\boldsymbol{w}\|^2} \\ \text{s.t.} \quad & y^{(n)}(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}^{(n)} + b) \geq 1, \forall n \in \{1, \cdots, N\}. \end{aligned} \tag{3.86}$$

数据集中所有满足 $y^{(n)}(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}^{(n)} + b) = 1$ 的样本点，都称为支持向量（Support Vector）.

对于一个线性可分的数据集，其分割超平面有很多个，但是间隔最大的超平面是唯一的. 图3.6给定了支持向量机的最大间隔分割超平面的示例，其中轮廓线加粗的样本点为支持向量.

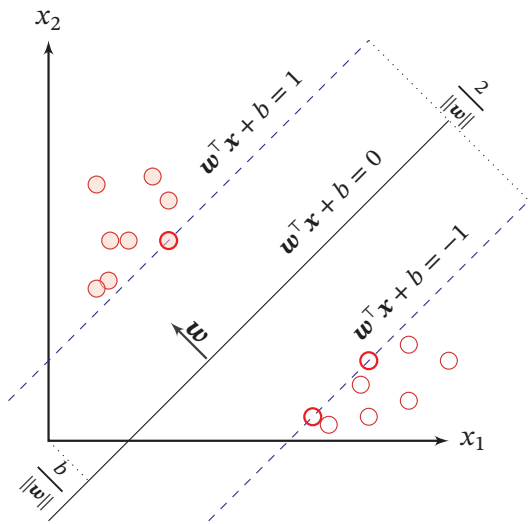


图 3.6 支持向量机示例

3.5.1 参数学习

为了找到最大间隔分割超平面，将公式(3.86)的目标函数写为凸优化问题

$$\begin{aligned} \min_{\boldsymbol{w},b} \quad & \frac{1}{2}\|\boldsymbol{w}\|^2 \\ \text{s.t.} \quad & 1 - y^{(n)}(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}^{(n)} + b) \leq 0, \qquad \forall n \in \{1, \cdots, N\}. \end{aligned} \tag{3.87}$$

使用拉格朗日乘数法，公式(3.87)的拉格朗日函数为

参见第C.3节.

$$\Lambda(\boldsymbol{w}, b, \lambda) = \frac{1}{2}\|\boldsymbol{w}\|^2 + \sum_{n=1}^{N} \lambda_n \Big(1 - y^{(n)}(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}^{(n)} + b)\Big), \tag{3.88}$$

其中 $\lambda_1 \geq 0, \cdots, \lambda_N \geq 0$ 为拉格朗日乘数. 计算 $\Lambda(\boldsymbol{w}, b, \lambda)$ 关于 $\boldsymbol{w}$ 和 b 的导数，并令其等于 0，得到

$$\boldsymbol{w} = \sum_{n=1}^{N} \lambda_n y^{(n)} \boldsymbol{x}^{(n)}, \tag{3.89}$$

$$0 = \sum_{n=1}^{N} \lambda_n y^{(n)}. \tag{3.90}$$

将公式(3.89)代入公式(3.88)，并利用公式(3.90)，得到拉格朗日对偶函数

$$\Gamma(\lambda) = -\frac{1}{2}\sum_{n=1}^{N}\sum_{m=1}^{N} \lambda_m \lambda_n y^{(m)} y^{(n)} (\boldsymbol{x}^{(m)})^{\mathrm{T}} \boldsymbol{x}^{(n)} + \sum_{n=1}^{N} \lambda_n. \tag{3.91}$$

支持向量机的主优化问题为凸优化问题，满足强对偶性，即主优化问题可以通过最大化对偶函数 $\max_{\lambda \geq 0} \Gamma(\lambda)$ 来求解. 对偶函数 $\Gamma(\lambda)$ 是一个凹函数，因此最大化对偶函数是一个凸优化问题，可以通过多种凸优化方法来进行求解，得到拉格朗日乘数的最优值 λ^*. 但由于其约束条件的数量为训练样本数量，一般的优化方法代价比较高，因此在实践中通常采用比较高效的优化方法，比如序列最小优化（Sequential Minimal Optimization，SMO）算法 [Platt, 1998] 等.

根据KKT条件中的互补松弛条件，最优解满足 $\lambda_n^* \big(1 - y^{(n)}(\boldsymbol{w}^{*\mathrm{T}}\boldsymbol{x}^{(n)} + b^*)\big) = 0$. 如果样本 $\boldsymbol{x}^{(n)}$ 不在约束边界上，$\lambda_n^* = 0$，其约束失效；如果样本 $\boldsymbol{x}^{(n)}$ 在约束边界上，$\lambda_n^* \geq 0$. 这些在约束边界上的样本点称为支持向量（Support Vector），即离决策平面距离最近的点.

参见公式(C.26).

在计算出 λ^* 后，根据公式(3.89)计算出最优权重 $\boldsymbol{w}^*$，最优偏置 b^* 可以通过任选一个支持向量 $(\tilde{\boldsymbol{x}}, \tilde{y})$ 计算得到：

$$b^* = \tilde{y} - \boldsymbol{w}^{*\mathrm{T}}\tilde{\boldsymbol{x}}. \tag{3.92}$$

最优参数的支持向量机的决策函数为

$$f(\boldsymbol{x}) = \text{sgn}(\boldsymbol{w}^{*\top}\boldsymbol{x} + b^*) \tag{3.93}$$

$$= \text{sgn}\left(\sum_{n=1}^{N} \lambda_n^* y^{(n)} (\boldsymbol{x}^{(n)})^\top \boldsymbol{x} + b^*\right). \tag{3.94}$$

支持向量机的决策函数只依赖于 $\lambda_n^* > 0$ 的样本点，即支持向量.

支持向量机的目标函数可以通过 SMO 等优化方法得到全局最优解，因此比其他分类器的学习效率更高. 此外，支持向量机的决策函数只依赖于支持向量，与训练样本总数无关，分类速度比较快.

3.5.2 核函数

支持向量机还有一个重要的优点是可以使用核函数（Kernel Function）隐式地将样本从原始特征空间映射到更高维的空间，并解决原始特征空间中的线性不可分问题. 比如在一个变换后的特征空间 ϕ 中，支持向量机的决策函数为

$$f(\boldsymbol{x}) = \text{sgn}(\boldsymbol{w}^{*\top}\phi(\boldsymbol{x}) + b^*) \tag{3.95}$$

$$= \text{sgn}\left(\sum_{n=1}^{N} \lambda_n^* y^{(n)} k(\boldsymbol{x}^{(n)}, \boldsymbol{x}) + b^*\right), \tag{3.96}$$

其中 $k(\boldsymbol{x}, \boldsymbol{z}) = \phi(\boldsymbol{x})^\top \phi(\boldsymbol{z})$ 为核函数. 通常我们不需要显式地给出 $\phi(\boldsymbol{x})$ 的具体形式，可以通过核技巧（Kernel Trick）来构造. 比如以 $\boldsymbol{x}, \boldsymbol{z} \in \mathbb{R}^2$ 为例，我们可以构造一个核函数：

$$k(\boldsymbol{x}, \boldsymbol{z}) = (1 + \boldsymbol{x}^\top \boldsymbol{z})^2 = \phi(\boldsymbol{x})^\top \phi(\boldsymbol{z}), \tag{3.97}$$

参见习题3-10.

来隐式地计算 $\boldsymbol{x}, \boldsymbol{z}$ 在特征空间 ϕ 中的内积，其中

$$\phi(\boldsymbol{x}) = [1, \sqrt{2}x_1, \sqrt{2}x_2, \sqrt{2}x_1 x_2, x_1^2, x_2^2]^\top. \tag{3.98}$$

3.5.3 软间隔

在支持向量机的优化问题中，约束条件比较严格. 如果训练集中的样本在特征空间中不是线性可分的，就无法找到最优解. 为了能够容忍部分不满足约束的样本，我们可以引入松弛变量（Slack Variable）ξ，将优化问题变为

$$\min_{\boldsymbol{w}, b} \quad \frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_{n=1}^{N} \xi_n \tag{3.99}$$

$$\begin{aligned} \text{s.t.} \quad & 1 - y^{(n)}(\boldsymbol{w}^{\top}\boldsymbol{x}^{(n)} + b) - \xi_n \leq 0, \qquad \forall n \in \{1, \cdots, N\} \\ & \xi_n \geq 0, \qquad \forall n \in \{1, \cdots, N\} \end{aligned}$$

其中参数 $C > 0$ 用来控制间隔和松弛变量惩罚的平衡. 引入松弛变量的间隔称为软间隔（Soft Margin）. 公式(3.99)也可以表示为经验风险+正则化项的形式：

$$\min_{\boldsymbol{w},b} \quad \sum_{n=1}^{N} \max\Big(0, 1 - y^{(n)}(\boldsymbol{w}^{\top}\boldsymbol{x}^{(n)} + b)\Big) + \frac{1}{2C}\|\boldsymbol{w}\|^2, \tag{3.100}$$

其中可以把 $\max\Big(0, 1 - y^{(n)}(\boldsymbol{w}^{\top}\boldsymbol{x}^{(n)} + b)\Big)$ 看作损失函数，称为Hinge 损失函数（Hinge Loss Function），把 $\frac{1}{2C}\|\boldsymbol{w}\|^2$ 看作正则化项，$\frac{1}{C}$ 是正则化系数.

参见公式(2.20).

软间隔支持向量机的参数学习和原始支持向量机类似，其最终决策函数也只和支持向量有关，即满足 $1 - y^{(n)}(\boldsymbol{w}^{\top}\boldsymbol{x}^{(n)} + b) - \xi_n = 0$ 的样本.

参见习题3-11.

3.6 损失函数对比

本章介绍了三种二分类模型：Logistic 回归、感知器和支持向量机. 虽然它们的决策函数相同，但由于使用了不同的损失函数以及相应的优化方法，导致它们在实际任务上的表现存在一定的差异.

为了比较这些损失函数，我们统一定义类别标签 $y \in \{+1, -1\}$，并定义 $f(\boldsymbol{x}; \boldsymbol{w}) = \boldsymbol{w}^{\top}\boldsymbol{x} + b$. 这样对于样本 $(\boldsymbol{x}, y)$，若 $yf(\boldsymbol{x}; \boldsymbol{w}) > 0$，则分类正确；若 $yf(\boldsymbol{x}; \boldsymbol{w}) < 0$，则分类错误. 这样，为了方便比较这些模型，我们可以将它们的损失函数都表述为定义在 $yf(\boldsymbol{x}; \boldsymbol{w})$ 上的函数.

Logistic 回归的损失函数可以改写为

$$\begin{aligned} \mathcal{L}_{LR} &= -I(y=1)\log\sigma\big(f(\boldsymbol{x};\boldsymbol{w})\big) - I(y=-1)\log\Big(1 - \sigma\big(f(\boldsymbol{x};\boldsymbol{w})\big)\Big) && (3.101) \\ &= -I(y=1)\log\sigma\big(f(\boldsymbol{x};\boldsymbol{w})\big) - I(y=-1)\log\sigma\big(-f(\boldsymbol{x};\boldsymbol{w})\big) && (3.102) \\ &= -\log\sigma\big(yf(\boldsymbol{x};\boldsymbol{w})\big) && (3.103) \\ &= \log\big(1 + \exp\big(-yf(\boldsymbol{x};\boldsymbol{w})\big)\big). && (3.104) \end{aligned}$$

$I(\cdot)$ 为指示函数.

$1 - \sigma(x) = \sigma(-x)$.

$y \in \{+1, -1\}$.

感知器的损失函数为

$$\mathcal{L}_p = \max\big(0, -yf(\boldsymbol{x};\boldsymbol{w})\big). \tag{3.105}$$

软间隔支持向量机的损失函数为

$$\mathcal{L}_{hinge} = \max\big(0, 1 - yf(\boldsymbol{x};\boldsymbol{w})\big). \tag{3.106}$$

平方损失可以重写为

$$\mathcal{L}_{squared} = \big(y - f(\boldsymbol{x};\boldsymbol{w})\big)^2 \tag{3.107}$$

$$= 1 - 2yf(\boldsymbol{x};\boldsymbol{w}) + (yf(\boldsymbol{x};\boldsymbol{w}))^2 \tag{3.108}$$

$$= \big(1 - yf(\boldsymbol{x};\boldsymbol{w})\big)^2. \tag{3.109}$$

$y^2 = 1.$

图3.7给出了不同损失函数的对比. 对于二分类来说，当 $yf(\boldsymbol{x};\boldsymbol{w}) > 0$ 时，分类器预测正确，并且 $yf(\boldsymbol{x};\boldsymbol{w})$ 越大，模型的预测越正确；当 $yf(\boldsymbol{x};\boldsymbol{w}) < 0$ 时，分类器预测错误，并且 $yf(\boldsymbol{x};\boldsymbol{w})$ 越小，模型的预测越错误. 因此，一个好的损失函数应该随着 $yf(\boldsymbol{x};\boldsymbol{w})$ 的增大而减少. 从图3.7中看出，除了平方损失，其他损失函数都比较适合于二分类问题.

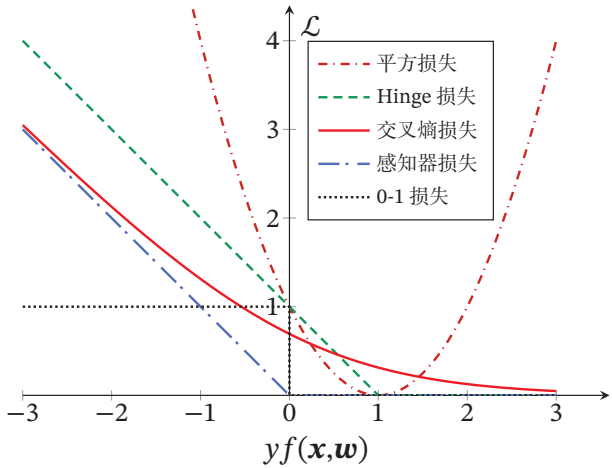


图 3.7 不同损失函数的对比

3.7 总结和深入阅读

和回归问题不同，分类问题中的目标标签 y 是离散的类别标签，因此分类问题中的决策函数需要输出离散值或是标签的后验概率. 线性分类模型一般是一个广义线性函数，即一个或多个线性判别函数加上一个非线性激活函数. 所谓“线性”是指决策边界由一个或多个超平面组成.

表3.1给出了几种常见的线性模型的比较. 在 Logistic 回归和 Softmax 回归中，$\boldsymbol{y}$ 为类别的 one-hot 向量表示；在感知器和支持向量机中，y 为 $\{+1, -1\}$.

表 3.1 几种常见的线性模型对比

线性模型	激活函数	损失函数	优化方法
线性回归	-	$(y-\boldsymbol{w}^{\top}\boldsymbol{x})^2$	最小二乘、梯度下降
Logistic 回归	$\sigma(\boldsymbol{w}^{\top}\boldsymbol{x})$	$\boldsymbol{y}\log\sigma(\boldsymbol{w}^{\top}\boldsymbol{x})$	梯度下降
Softmax 回归	$\text{softmax}(\boldsymbol{W}^{\top}\boldsymbol{x})$	$\boldsymbol{y}\log\text{softmax}(\boldsymbol{W}^{\top}\boldsymbol{x})$	梯度下降
感知器	$\text{sgn}(\boldsymbol{w}^{\top}\boldsymbol{x})$	$\max(0,-y\boldsymbol{w}^{\top}\boldsymbol{x})$	随机梯度下降
支持向量机	$\text{sgn}(\boldsymbol{w}^{\top}\boldsymbol{x})$	$\max(0,1-y\boldsymbol{w}^{\top}\boldsymbol{x})$	二次规划、SMO 等

Logistic 回归是一种概率模型，其通过使用 Logistic 函数来将一个实数值映射到 $[0,1]$ 之间. 事实上，还有很多函数也可以达到此目的，比如正态分布的累积概率密度函数，即 probit 函数. 这些知识可以参考《Pattern Recognition and Machine Learning》[Bishop, 2007] 的第 4 章.

感知器作为一种最简单的神经网络，其学习算法也非常直观有效 [Rosenblatt, 1958]. [Freund et al., 1999] 提出了使用核技巧改进感知器学习算法，并用投票感知器来提高泛化能力. [Collins, 2002] 将感知器算法扩展到结构化学习，给出了相应的收敛性证明，并且提出一种更加有效并且实用的参数平均化策略.

要深入了解支持向量机以及核方法，可以参考文献《Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond》[Scholkopf et al., 2001].

习题

习题 3-1 证明在两类线性分类中，权重向量 $\boldsymbol{w}$ 与决策平面正交.

习题 3-2 在线性空间中，证明一个点 $\boldsymbol{x}$ 到平面 $f(\boldsymbol{x};\boldsymbol{w})=\boldsymbol{w}^{\top}\boldsymbol{x}+b=0$ 的距离为 $|f(\boldsymbol{x};\boldsymbol{w})|/\|\boldsymbol{w}\|$.

习题 3-3 在线性分类中，决策区域是凸的. 即若点 $\boldsymbol{x}_1$ 和 $\boldsymbol{x}_2$ 被分为类别 c，则点 $\rho\boldsymbol{x}_1+(1-\rho)\boldsymbol{x}_2$ 也会被分为类别 c，其中 $\rho\in(0,1)$.

习题 3-4 给定一个多分类的数据集，证明：1）如果数据集中每个类的样本都和除该类之外的样本是线性可分的，则该数据集一定是线性可分的；2）如果数据集中每两个类的样本是线性可分的，则该数据集不一定是线性可分的.

习题 3-5 在 Logistic 回归中，是否可以用 $\hat{y}=\sigma(\boldsymbol{w}^{\top}\boldsymbol{x})$ 去逼近正确的标签 y，并用平方损失 $(y-\hat{y})^2$ 最小化来优化参数 $\boldsymbol{w}$?

习题 3-6 在 Softmax 回归的风险函数（公式 (3.39)）中，如果加上正则化项会有什么影响？

习题 3-7 验证平均感知器训练算法3.2中给出的平均权重向量的计算方式和公式 (3.77) 等价.

习题 3-8 证明定理3.2.

习题 3-9 若数据集线性可分，证明支持向量机中将两类样本正确分开的最大间隔分割超平面存在且唯一.

习题 3-10 验证公式 (3.97).

习题 3-11 在软间隔支持向量机中，试给出原始优化问题的对偶问题，并列出其 KKT 条件.

参考文献

Bishop C M, 2007. Pattern recognition and machine learning[M]. 5th edition. Springer.

Collins M, 2002. Discriminative training methods for hidden markov models: Theory and experiments with perceptron algorithms[C]//Proceedings of the conference on Empirical methods in natural language processing. 1-8.

Daumé III H, 2012. A course in machine learning[EB/OL]. http://ciml.info/.

Freund Y, Schapire R E, 1999. Large margin classification using the perceptron algorithm[J]. Machine learning, 37(3):277-296.

Novikoff A B, 1963. On convergence proofs for perceptrons[R]. DTIC Document.

Platt J, 1998. Sequential minimal optimization: A fast algorithm for training support vector machines[R]. 21.

Rosenblatt F, 1958. The perceptron: a probabilistic model for information storage and organization in the brain.[J]. Psychological review, 65(6):386.

Scholkopf B, Smola A J, 2001. Learning with kernels: support vector machines, regularization, optimization, and beyond[M]. MIT press.

第二部分

基础模型